EAST WEST UNIVERSITY

Term Project

Theoretical vs. Empirical Analysis of Algorithms

Course Code: CSE246

Course Title: Algorithms

Section: 05

Submitted To

Dr. Hasan Mahmood Aminul Islam

Assistant Professor, Dept of CSE

East West University

Submitted By

Name	ID
Fayaza Islam	2023-3-60-314
Pulok Akibuzzaman	2023-3-60-051
Sayeda Fariha	2023-3-60-567

Date of submission: 4th September, 2025

Content Table

Introduction:

This term project investigates the relationship between the theoretical complexity of algorithms and their actual performance when executed on real data. While Big-O notation provides a mathematical framework for analyzing algorithm efficiency in terms of time and space, it does not account for practical factors such as hardware constraints, compiler behavior, and input characteristics. To address this gap, the project involves implementing ten widely-used algorithms in C++, analyzing their theoretical complexities, and empirically measuring their execution times across various input sizes.

The objective is to observe how execution time scales with input size and to compare these results with theoretical expectations. Each algorithm is evaluated in terms of worst-case time and space complexity, followed by empirical benchmarking using input sizes ranging from 1 to 10,000. The results are visualized using performance graphs with logarithmic scaling to highlight differences in growth rates. This approach enables a clear comparison of algorithmic behavior in both theoretical and practical contexts, providing a comprehensive understanding of their efficiency and limitations.

Algorithm Analysis

1. Linear Search

Time Complexity (Worst Case): O(n)

In the worst-case scenario, Linear Search algorithm examines every element in the array to find the target (or determine it's not present). This results in a time complexity of O(n), where n is the number of elements in the input array.

Space Complexity: O(1)

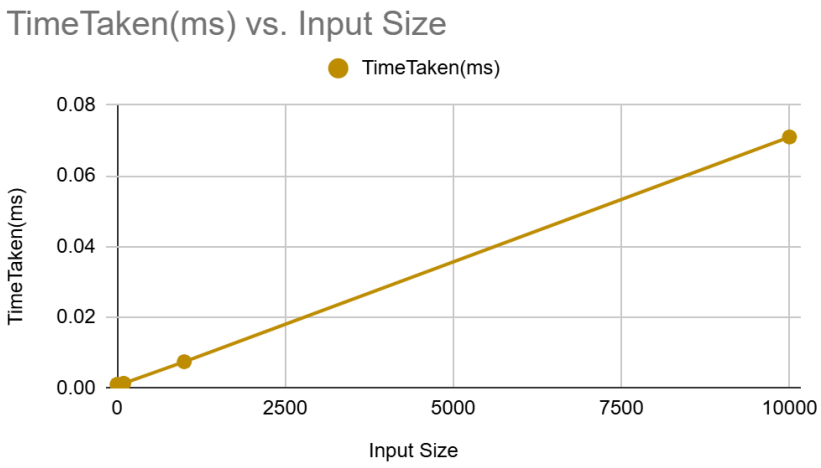
Linear Search does not require any additional data structures or memory allocation beyond a few variables for indexing and comparison. Therefore, its space complexity is O(1), meaning it uses constant space regardless of input size.

Table:

Input Size	TimeTaken(ms)
1	0.0011
10	0.0006
100	0.0014
1000	0.0075
10000	0.071

Table 1: Execution table of Linear Search Algorithm

Graph:



Graph 1: Time complexity of Linear Search Algorithm

2. Binary Search

Time Complexity (Worst Case): O(log n)

Binary Search repeatedly divides the sorted array in half to locate the target element. Each division reduces the search space by half, resulting in a logarithmic time complexity of O(log n).

Space Complexity: O(1)

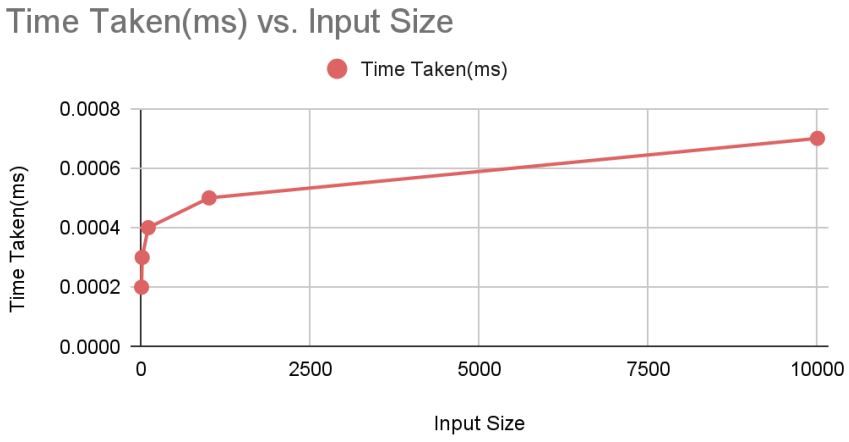
Space Complexity: The algorithm uses a constant number of variables (left, right, mid) and does not require any additional memory structures, so the space complexity is O(1).

Table:

Input Size	Time Taken(ms)
1	0.0002
10	0.0003
100	0.0004
1000	0.0005
10000	0.0007

Table 2: Execution table of Binary Search Algorithm

Graph:



Graph 2: Time complexity of Binary Search Algorithm

3. Merge Sort

Time Complexity (Worst & Average Case): O(n log n)

Merge Sort is a divide-and-conquer algorithm that recursively splits the array into halves and merges them in sorted order. Each level of recursion processes all elements, and there are log n levels, resulting in O(n log n) time complexity.

Space Complexity: O(n)

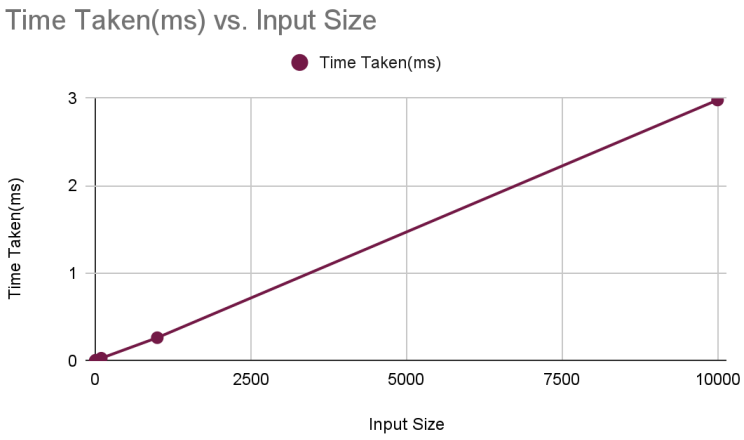
Merge Sort requires additional space for temporary arrays during the merge step. These arrays store parts of the original array, leading to O(n) space usage.

Table:

Input Size	Time Taken(ms)
1	0.0001
10	0.0094
100	0.0318
1000	0.266
10000	2.978

Table 3: Execution table of Merge sort

Graph:



Graph 3: Time complexity of Merge sort

4. Quick Sort

Time Complexity (Worst Case): $O(n^2)$

If the pivot selection is poor (e.g., always the smallest or largest element), the partitions become highly unbalanced, degrading performance to $O(n^2)$.

Space Complexity: O(log n) (due to recursive stack space)

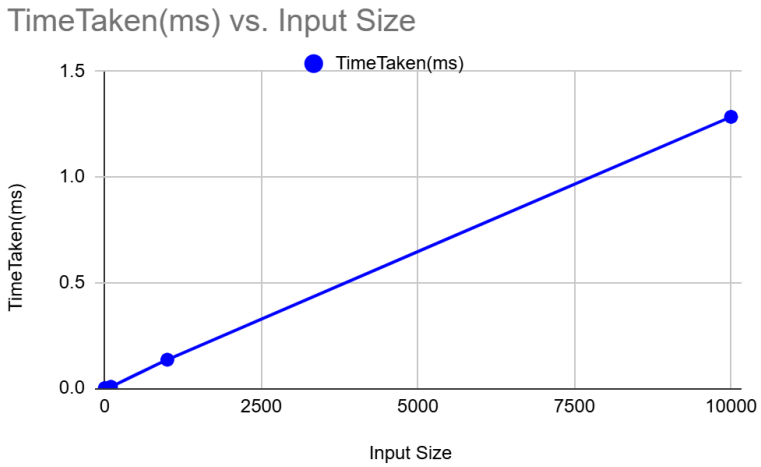
The algorithm is in-place, but recursive calls use stack space. In the average case, the recursion depth is log n, so space complexity is O(log n).

Table:

Input Size	TimeTaken(ms)
1	0
10	0.0007
100	0.0067
1000	0.1348
10000	1.2837

Table 4: Execution table of Quick sort

Graph:



Graph 4: Time complexity of Quick sort

5. Recursive Fibonacci

Time Complexity (Average/Worst Case): $O(2^n)$

The recursive Fibonacci function makes two recursive calls for each non-base case. This leads to a binary tree of calls, where the number of calls grows exponentially with n. Hence, the time complexity is $O(2^n)$.

Space Complexity: O(n)

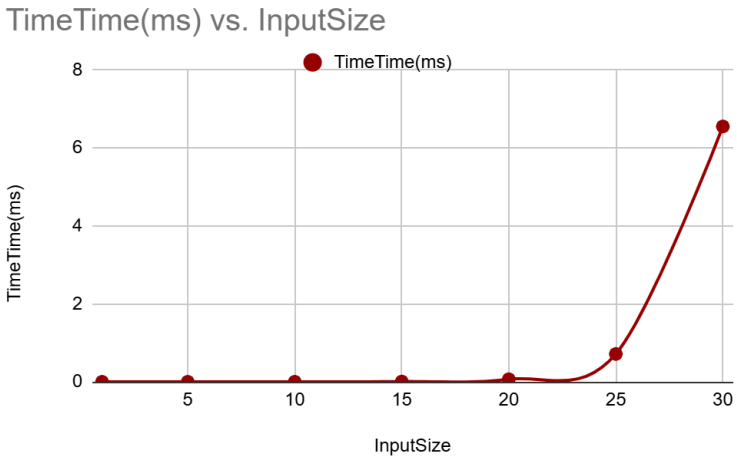
Space Complexity: The maximum depth of the recursion stack is n, as each call waits for the result of two others before returning. Therefore, the space complexity is O(n).

Table:

Input Size	TimeTaken(ms)
1	0
5	0.0003
10	0.0011
15	0.0069
20	0.0656
25	0.713
30	6.5474

Table 5: Execution table of Recursive Fibonacci

Graph:



Graph 5: Time complexity of Recursive Fibonacci

6. 0/1 Knapsack (Dynamic Programming)

Time Complexity: O(nW); Here, (n = number of items, W = maximum weight capacity).

The algorithm builds a 2D table dp[n+1][W+1], where each cell is computed based on previous values. Since it iterates over all items and all weight capacities, the time complexity is O(nW).

Space Complexity: O(nW)

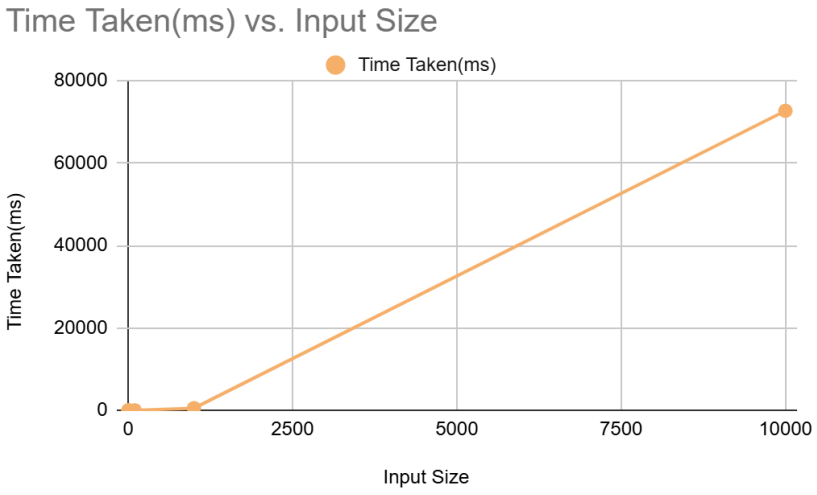
The same 2D table requires O(nW) space to store intermediate results. This can be optimized to O(W) using a 1D array, but your current implementation uses the full table.

Table:

Input Size	Time Taken(ms)
1	0.0017
10	0.0596
100	5.4873
1000	527.152
10000	72684.6

Table 6: Execution table of Knapsack

Graph:



Graph 6: Time complexity of Knapsack

7. Breadth-First Search (BFS)

Time Complexity: O(V + E); Here, (V = number of vertices, E = number of edges).

BFS visits each vertex once and explores all adjacent edges. The total number of operations is proportional to the number of vertices plus the number of edges, resulting in O(V + E) time complexity.

Space Complexity: O(V)

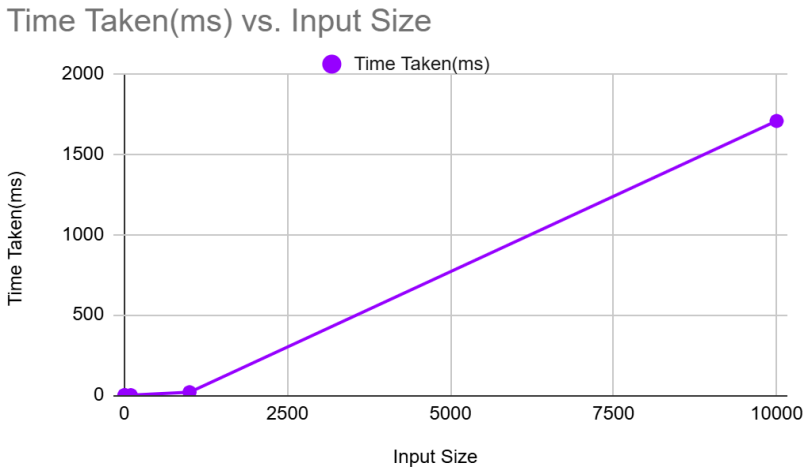
BFS uses a visited array and a queue to manage traversal, both of which require space proportional to the number of vertices, hence O(V) space complexity.

Table:

Input Size	Time Taken(ms)
1	0.0024
10	0.0044
100	0.1745
1000	18.5821
10000	1707.09

Table 7: Execution table of BFS

Graph:



Graph 7: Time complexity of BFS

8. Dijkstra's Algorithm

Time Complexity: O((V + E) log V); Here, (V = number of vertices, E = number of edges). Dijkstra's algorithm uses a priority queue to efficiently select the next vertex with the smallest tentative distance. Each vertex and edge may be processed, and each priority queue operation takes O(log V) time. This results in a total time complexity of O((V + E) log V).

Space Complexity: O(V + E)

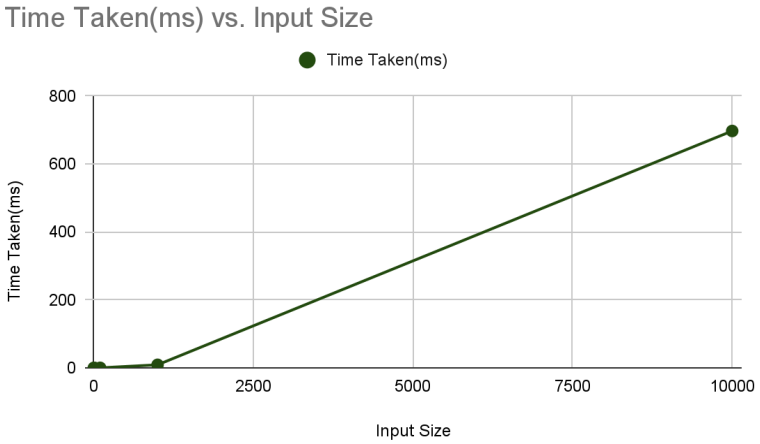
The graph is stored as an adjacency list (O(V + E)), and additional arrays like dist and the priority queue also contribute to space usage, but remain within the same bound.

Table:

Input Size	Time Taken(ms)
1	0.0032
10	0.0101
100	0.2127
1000	9.0844
10000	696.888

Table 8: Execution table of Dijkstra's Algorithm

Graph:



Graph 8: Time complexity of Dijkstra's Algorithm

9. Prim's Algorithm

Time Complexity: O(E log V); Here, (E = number of edges, V = number of vertices)

Prim’s algorithm uses a priority queue (min-heap) to efficiently select the next minimum-weight edge. Each edge may be pushed into the queue, and each operation takes O(log V) time. Thus, the total time complexity is O(E log V).

Space Complexity: O(V + E)

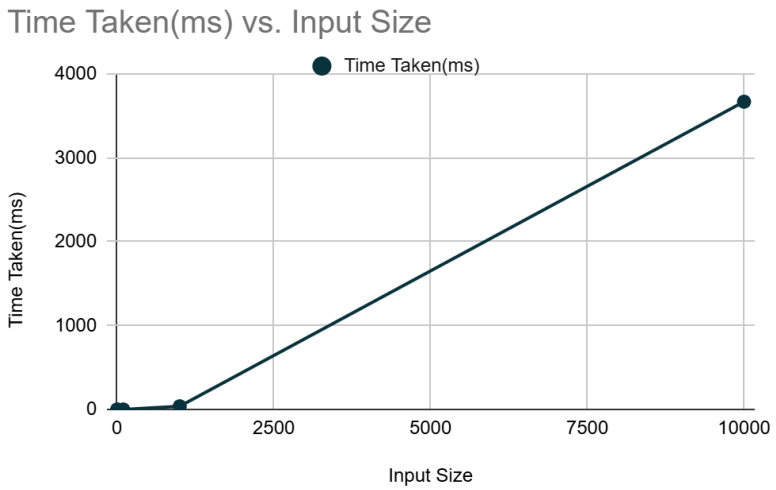
The graph is stored as an adjacency list, which takes O(V + E) space. Additional arrays like key, inMST, and the priority queue also contribute to space usage but remain within O(V + E).

Table:

Input Size	Time Taken(ms)
1	0.032
10	0.0191
100	0.6034
1000	38.7333
10000	3668.73

Table 9: Execution table of Prim’s Algorithm

Graph:



Graph 9: Time complexity of Prim's Algorithm

10. Floyd-Warshall Algorithm

Time Complexity: $O(n^3)$

Floyd-Warshall uses three nested loops to update the shortest paths between all pairs of vertices. Each loop runs n times, resulting in $O(n^3)$ time complexity.

Space Complexity: $O(n^2)$

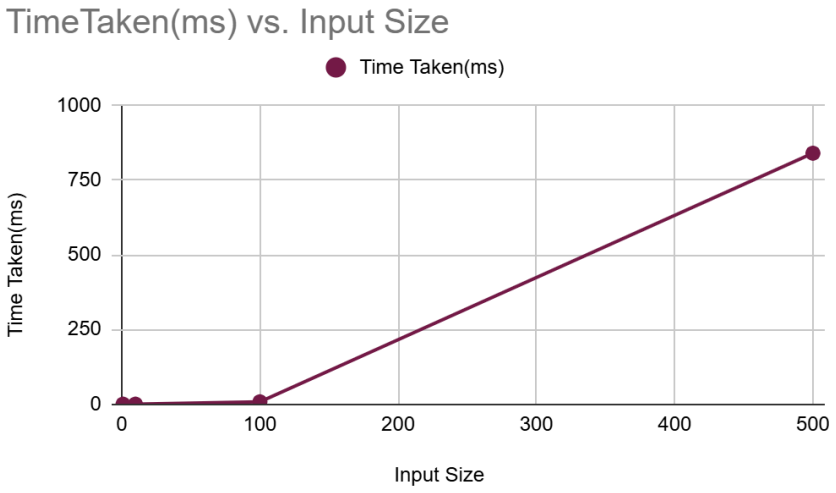
The algorithm maintains a 2D matrix dist[n][n] to store shortest distances between every pair of vertices, leading to $O(n^2)$ space usage.

Table:

Input Size	Time Taken(ms)
1	0.002
10	0.014
100	7.8441
500	840.144

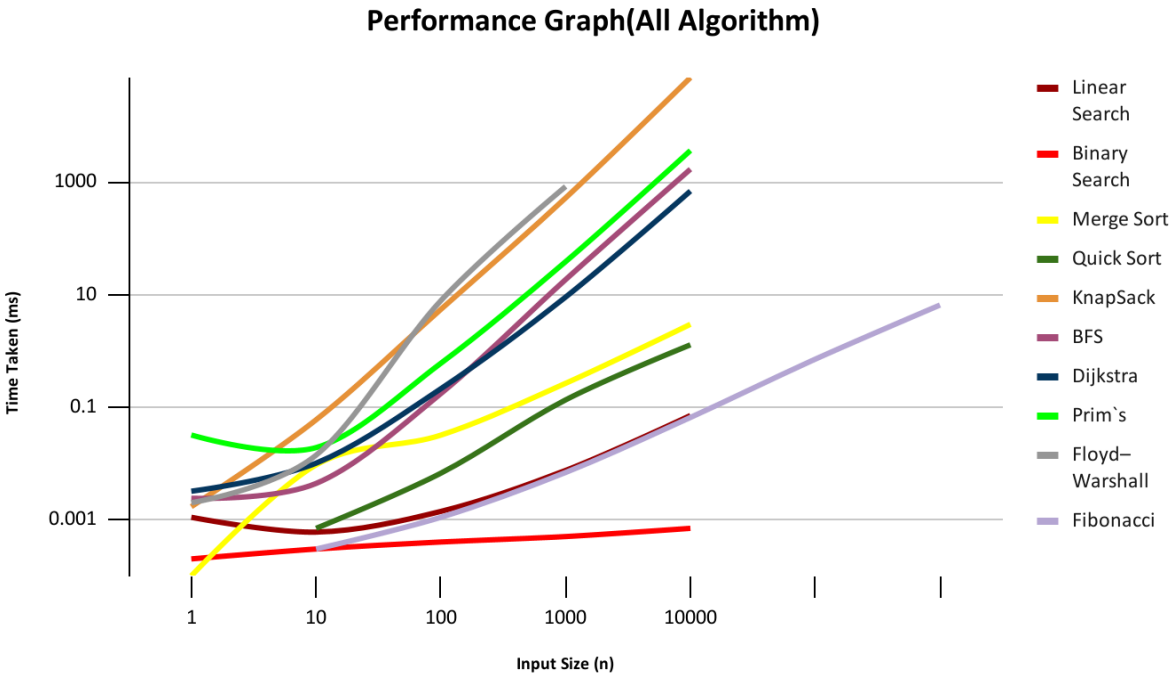
Table 10: Execution table of Floyd-Warshall Algorithm

Graph:



Graph 10: Time complexity of Floyd-Warshall Algorithm

Performance Graph:



Graph 11: Performance Graph For All Algorithms

Conclusion & Discussion:

The empirical results obtained from testing various algorithms generally align with their theoretical time complexities. For instance, the graph of Floyd-Warshall, which has a time complexity of $O(n^3)$, shows a significantly steeper curve compared to Merge Sort, which operates in O(n log n) time. This steepness reflects the rapid growth in execution time as input size increases, confirming the inefficiency of cubic-time algorithms for large datasets.

In some cases, discrepancies were observed for smaller input sizes. For example, Linear Search occasionally performed faster than Binary Search for very small arrays due to lower constant overhead and simpler logic. Similarly, Quick Sort outperformed Merge Sort in some instances because of its in-place sorting and reduced memory usage, despite both having O(n log n) average-case complexity.

Algorithms with high polynomial or exponential complexity, such as Recursive Fibonacci ($O(2^n)$) and 0/1 Knapsack (O(nW)), demonstrated severe performance degradation as input size grew. These results highlight the practical limitations of such algorithms, especially in real-time or resource-constrained environments. While they are valuable for theoretical understanding and small-scale problems, they are often impractical for large datasets without optimization or alternative approaches.

Overall, this project emphasizes the importance of both theoretical analysis and empirical validation in algorithm design and selection. It also reinforces the need to consider input characteristics, implementation details, and system constraints when evaluating algorithmic performance.

Appendix

1. Linear Search: Searching for an element in an unsorted array.

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>

int linearSearch(const std::vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); ++i) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}
int main() {
    std::ifstream file("Linear_Search_input_1.txt");
    if (!file) {
        std::cerr << "Failed to open input file.\n";
        return 1;
    }
    std::vector<int> arr;
    int num;
    while (file >> num) {
        arr.push_back(num);
    }
    int target;
    std::cout << "Enter the number to search for: ";
    std::cin >> target;

    auto start = std::chrono::high_resolution_clock::now();
    int index = linearSearch(arr, target);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    if (index != -1)
        std::cout << "Element found at index " << index << ".\n";
    else
        std::cout << "Element not found.\n";

    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";
    return 0;
}
```

Output:

```
Enter the number to search for: 6356
Element found at index 0.
Time taken: 0.0011 milliseconds.
PS C:\Users\User\Downloads\New folder\
```

```
Enter the number to search for: 8252
Element found at index 9.
Time taken: 0.0006 milliseconds.
PS C:\Users\User\Downloads\New folder\Linear
```

```
Enter the number to search for: 3178
Element found at index 99.
Time taken: 0.0014 milliseconds.
PS C:\Users\User\Downloads\New folder\Linear
```

```
Enter the number to search for: 8928
Element found at index 999.
Time taken: 0.0075 milliseconds.
PS C:\Users\User\Downloads\New folder\Linear
```

```
Enter the number to search for: 99199
Element found at index 9999.
Time taken: 0.071 milliseconds.
PS C:\Users\User\Downloads\New folder\Linear
```

2. Binary Search: Searching for an element in a sorted array.

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <algorithm>

int binarySearch(const std::vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

```
int main() {
    std::ifstream file("Binary_Search_input_10000.txt");
    if (!file) {
        std::cerr << "Failed to open input file.\n";
        return 1;
    }

    std::vector<int> arr;
    int num;
    while (file >> num) {
        arr.push_back(num);
    }

    std::sort(arr.begin(), arr.end());
    int target;
    std::cout << "Enter the number to search for: ";
    std::cin >> target;

    auto start = std::chrono::high_resolution_clock::now();
    int index = binarySearch(arr, target);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    if (index != -1)
        std::cout << "Element found at index " << index << ".\n";
    else
        std::cout << "Element not found.\n";

    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:

```
Enter the number to search for: 5275
Element found at index 0.
Time taken: 0.0002 milliseconds.
PS F:\EWU\CSE246\Term Project CSE246\Binary Search>
Enter the number to search for: 8896
Element found at index 9.
Time taken: 0.0003 milliseconds.
PS F:\EWU\CSE246\Term Project CSE246\Binary Search>
Enter the number to search for: 9943
Element found at index 99.
Time taken: 0.0004 milliseconds.
PS F:\EWU\CSE246\Term Project CSE246\Binary Search>
Enter the number to search for: 9990
Element found at index 999.
Time taken: 0.0005 milliseconds.
PS F:\EWU\CSE246\Term Project CSE246\Binary Search>
Enter the number to search for: 99998
Element found at index 9999.
Time taken: 0.0007 milliseconds.
```

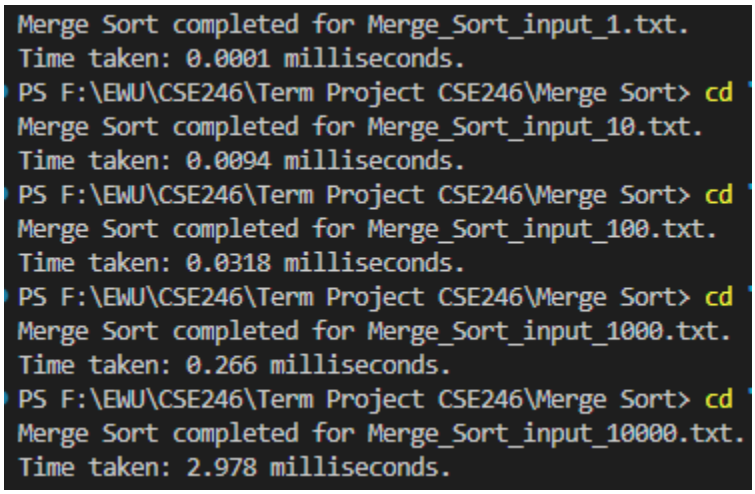
3. Merge Sort: A divide-and-conquer sorting algorithm.

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <string>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; ++i)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; ++j)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    std::string filename = "Merge_Sort_input_10000.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    std::vector<int> arr;
```

```
    int num;
    while (file >> num) {
        arr.push_back(num);
    }

    auto start = std::chrono::high_resolution_clock::now();
    mergeSort(arr, 0, arr.size() - 1);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "Merge Sort completed for " << filename << ".\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



4. Quick Sort: A divide-and-conquer sorting algorithm. (Analyze its average-case complexity, but be aware of the worst-case).

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>

int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
```

```
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    const std::string filename = "Quick_Sort_input_1.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    std::vector<int> arr;
    int num;
    while (file >> num) {
        arr.push_back(num);
    }

    std::cout << "Running Quick Sort on file: " << filename << "\n";

    auto start = std::chrono::high_resolution_clock::now();
    quickSort(arr, 0, arr.size() - 1);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "Quick Sort completed.\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:

```
Running Quick Sort on file: Quick_Sort_input_1.txt
Quick Sort completed.
Time taken: 0 milliseconds.
PS C:\Users\User\Downloads\New folder\Term Project CSE246 C-
Running Quick Sort on file: Quick_Sort_input_10.txt
Quick Sort completed.
Time taken: 0.0007 milliseconds.
PS C:\Users\User\Downloads\New folder\Term Project CSE246 C-
Running Quick Sort on file: Quick_Sort_input_100.txt
Quick Sort completed.
Time taken: 0.0067 milliseconds.
PS C:\Users\User\Downloads\New folder\Term Project CSE246 C-
Running Quick Sort on file: Quick_Sort_input_1000.txt
Quick Sort completed.
Time taken: 0.1348 milliseconds.
PS C:\Users\User\Downloads\New folder\Term Project CSE246 C-
Running Quick Sort on file: Quick_Sort_input_10000.txt
Quick Sort completed.
Time taken: 1.2837 milliseconds.
PS C:\Users\User\Downloads\New folder\Term Project CSE246 C-
```

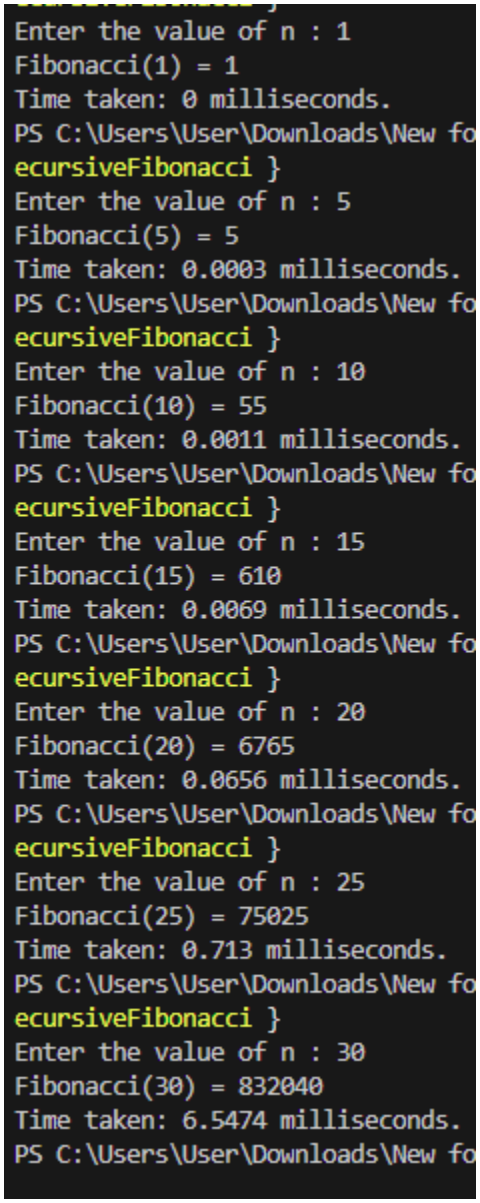
5. Recursive Fibonacci: The classic recursive implementation to find the n-th Fibonacci number. Note the exponential complexity.

Code:

```
#include <iostream>
#include <chrono>

int recursiveFibonacci(int n) {
    if (n <= 1)
        return n;
    return recursiveFibonacci(n - 1) + recursiveFibonacci(n - 2);
}

int main() {
    int n;
    std::cout << "Enter the value of n : ";
    std::cin >> n;

    if (n > 30) {
        std::cout << "Warning: Recursive Fibonacci is very slow for large n due to exponential complexity.\n";
    }

    auto start = std::chrono::high_resolution_clock::now();
```

```
    int result = recursiveFibonacci(n);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "Fibonacci(" << n << ") = " << result << "\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



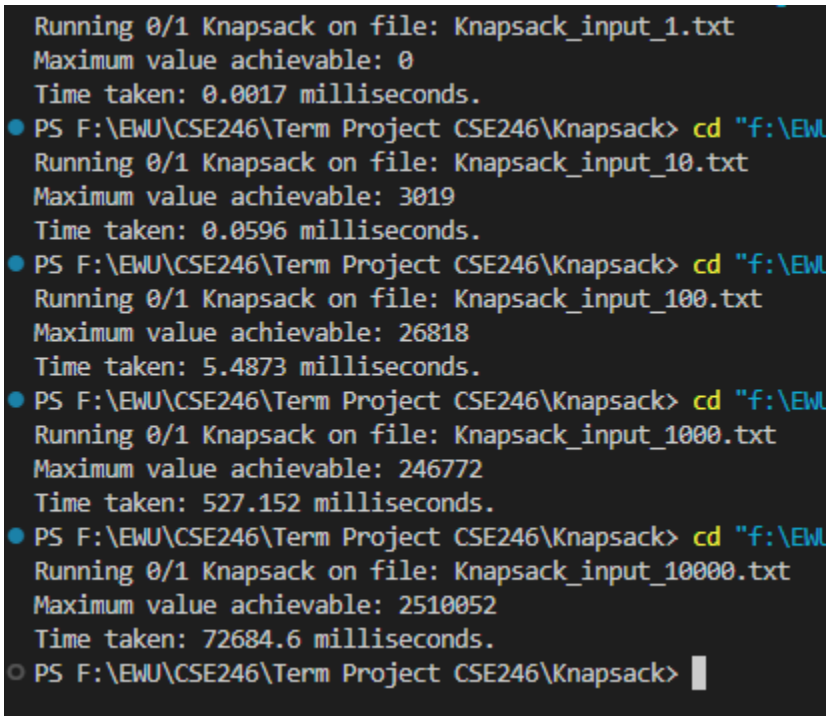
6. 0/1 Knapsack (Dynamic Programming): Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>

int knapsack(int W, const std::vector<int>& weights, const std::vector<int>& values, int n) {
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weights[i - 1] <= w)
                dp[i][w] = std::max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][W];
}

int main() {
    const std::string filename = "Knapsack_input_1.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    int n, W;
    file >> n >> W;

    std::vector<int> weights(n), values(n);
    for (int i = 0; i < n; i++) {
        file >> weights[i] >> values[i];
    }

    std::cout << "Running 0/1 Knapsack on file: " << filename << "\n";

    auto start = std::chrono::high_resolution_clock::now();
    int maxValue = knapsack(W, weights, values, n);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;
```

```
    std::cout << "Maximum value achievable: " << maxValue << "\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



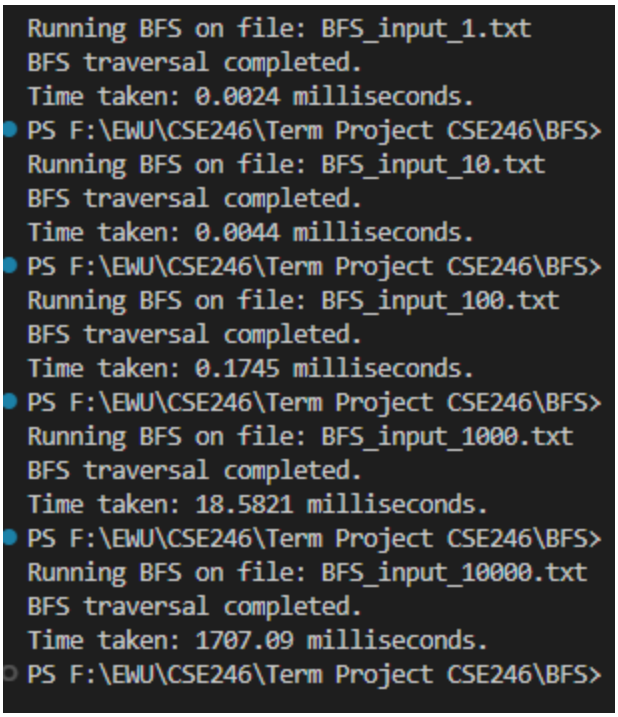
7. Breadth-First Search (BFS): Traversing a graph

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <queue>
#include <chrono>

void bfs(const std::vector<std::vector<int>>& graph, int start) {
    int n = graph.size();
    std::vector<bool> visited(n, false);
    std::queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
```

```
        int node = q.front();
        q.pop();

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

int main() {
    const std::string filename = "BFS_input_10000.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    int n;
    file >> n;

    std::vector<std::vector<int>> graph(n);
    int weight;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            file >> weight;
            if (weight != 0) {
                graph[i].push_back(j);
            }
        }
    }

    std::cout << "Running BFS on file: " << filename << "\n";

    auto start = std::chrono::high_resolution_clock::now();
    bfs(graph, 0);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "BFS traversal completed.\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



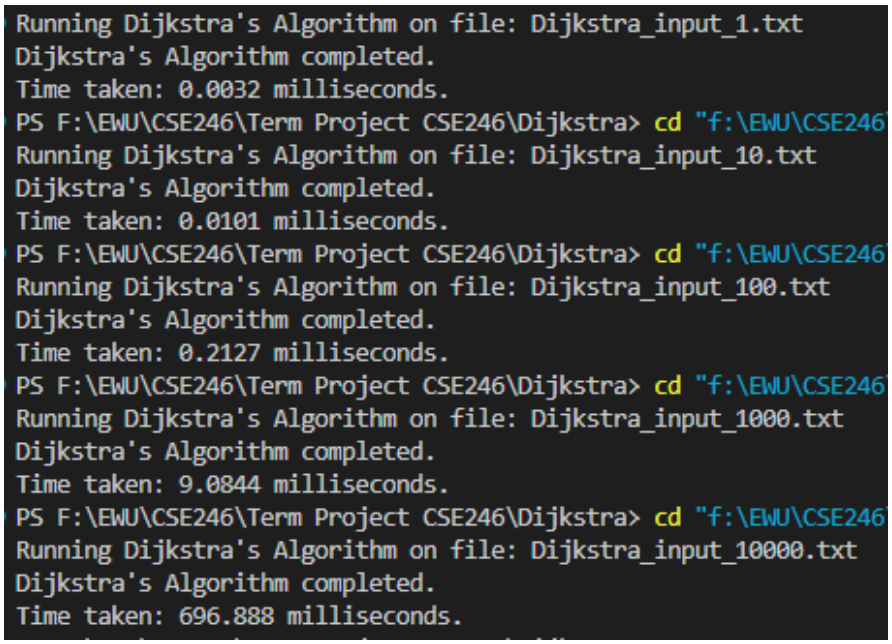
8. Dijkstra's Algorithm: Finding the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <queue>
#include <chrono>
#include <limits>

const int INF = std::numeric_limits<int>::max();

void dijkstra(const std::vector<std::vector<std::pair<int, int>>>& graph, int source, std::vector<int>& dist) {
    int n = graph.size();
    dist.assign(n, INF);
    dist[source] = 0;

    using pii = std::pair<int, int>;
    std::priority_queue<pii, std::vector<pii>, std::greater<pii>> pq;
    pq.push({0, source});

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
```

```
        pq.pop();

        if (d > dist[u]) continue;

        for (const auto& edge : graph[u]) {
            int v = edge.first;
            int weight = edge.second;

            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }
}
int main() {
    const std::string filename = "Dijkstra_input_10000.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    int n;
    file >> n;

    std::vector<std::vector<std::pair<int, int>>> graph(n);
    int weight;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            file >> weight;
            if (weight != 0) {
                graph[i].emplace_back(j, weight);
            }
        }
    }

    std::cout << "Running Dijkstra's Algorithm on file: " << filename << "\n";

    std::vector<int> dist;
    auto start = std::chrono::high_resolution_clock::now();
    dijkstra(graph, 0, dist);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "Dijkstra's Algorithm completed.\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



9. Prim's Algorithm: Finding a Minimum Spanning Tree (MST) for a weighted undirected graph.

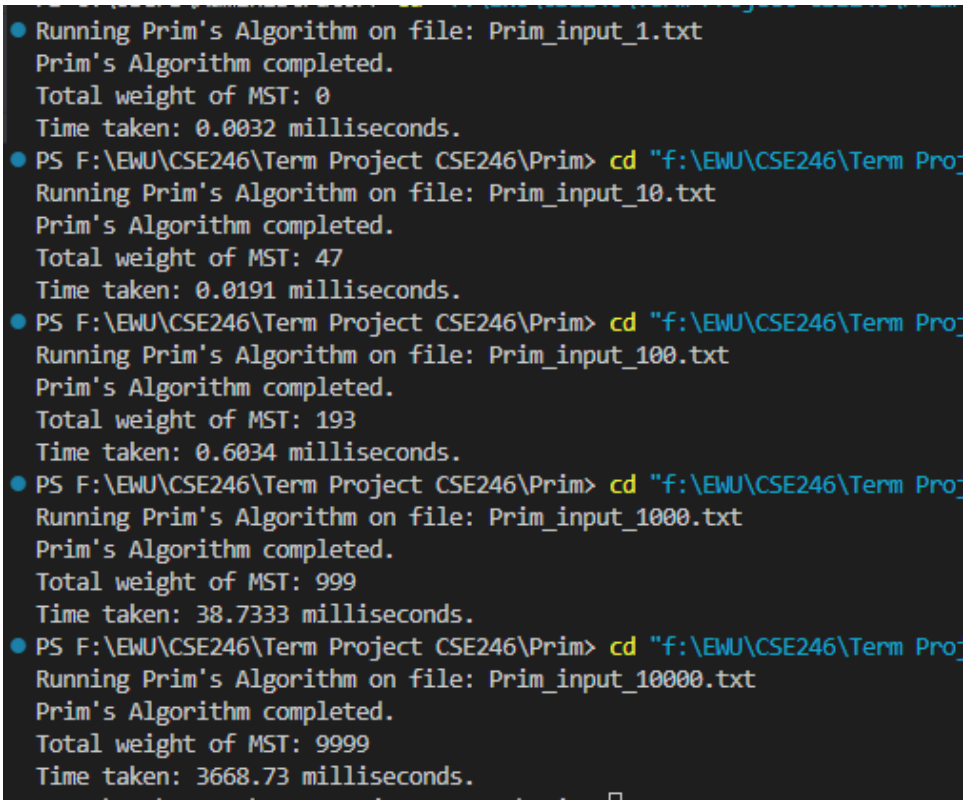
Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <limits>
#include <queue>

const int INF = std::numeric_limits<int>::max();

int primMST(const std::vector<std::vector<std::pair<int, int>>>& graph) {
    int n = graph.size();
    std::vector<int> key(n, INF);
    std::vector<bool> inMST(n, false);
    key[0] = 0;

    using pii = std::pair<int, int>;
    std::priority_queue<pii, std::vector<pii>, std::greater<pii>> pq;
    pq.push({0, 0});

    int totalWeight = 0;
```

```
    while (!pq.empty()) {
        int u = pq.top().second;
        int weight = pq.top().first;
        pq.pop();

        if (inMST[u]) continue;
        inMST[u] = true;
        totalWeight += weight;

        for (const auto& edge : graph[u]) {
            int v = edge.first;
            int w = edge.second;
            if (!inMST[v] && w < key[v]) {
                key[v] = w;
                pq.push({key[v], v});
            }
        }
    }

    return totalWeight;
}

int main() {
    const std::string filename = "Prim_input_10000.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open input file: " << filename << "\n";
        return 1;
    }

    int n;
    file >> n;

    std::vector<std::vector<std::pair<int, int>>> graph(n);
    int weight;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            file >> weight;
            if (weight != 0) {
                graph[i].emplace_back(j, weight);
                graph[j].emplace_back(i, weight);
            }
        }
    }

    std::cout << "Running Prim's Algorithm on file: " << filename << "\n";

    auto start = std::chrono::high_resolution_clock::now();
    int mstWeight = primMST(graph);
    auto end = std::chrono::high_resolution_clock::now();
```

```
    std::chrono::duration<double, std::milli> duration = end - start;

    std::cout << "Prim's Algorithm completed.\n";
    std::cout << "Total weight of MST: " << mstWeight << "\n";
    std::cout << "Time taken: " << duration.count() << " milliseconds.\n";

    return 0;
}
```

Output:



10. Floyd-Warshall Algorithm: Finding the shortest paths between all pairs of vertices in a weighted graph.

Code:

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>
#include <string>

const int INF = 1e9;
```

```
void floydWarshall(std::vector<std::vector<int>>& dist, int n) {
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
int main() {
    std::string filename = "Floyd_Warshall_input_1.txt";
    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Failed to open file: " << filename << "\n";
        return 1;
    }

    int n;
    file >> n;

    std::vector<std::vector<int>> graph(n, std::vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            file >> graph[i][j];

    std::cout << "Running Floyd-Warshall Algorithm on file: " << filename << "\n";

    auto start = std::chrono::high_resolution_clock::now();
    floydWarshall(graph, n);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration = end - start;
    std::cout << "Time taken: " << duration.count() << " ms\n";

    return 0;
}
```

Output:

```
Running Floyd-Warshall Algorithm on file: Floyd_Warshall_input_1.txt
Time taken: 0.0002 ms
PS F:\EWU\CSE246\Term Project CSE246\Floyd Warshall> cd "f:\EWU\CSE246\Term
Running Floyd-Warshall Algorithm on file: Floyd_Warshall_input_10.txt
Time taken: 0.014 ms
PS F:\EWU\CSE246\Term Project CSE246\Floyd Warshall> cd "f:\EWU\CSE246\Term
Running Floyd-Warshall Algorithm on file: Floyd_Warshall_input_100.txt
Time taken: 7.8441 ms
```

```
Running Floyd-Warshall Algorithm on file: Floyd_Warshall_input_500.txt
Time taken: 840.144 ms
PS F:\EWU\CSE246\Term Project CSE246\Floyd Warshall>
```